

CS5070 Mathematical Structures for Computer Science

Formal Language

Kun Suo

Department of Computer Science
College of Computing and Software Engineering
Kennesaw State University

ksuo@kennesaw.edu

How This Lecture Connects to Previous Topics

Previous Topics	Application in Programming Languages
Logic & Proofs	Program correctness, assertions, axiomatic semantics
Sets	Languages as sets of valid strings
Relations & Functions	Semantic mappings, variable lookup, denotational semantics
Sequences & Recursion	Recursive grammar rules and derivations
Induction	Loop invariants and correctness proofs
Graphs & Trees	Parse trees and syntax trees
Counting	Number of possible strings and grammar structures

Syntax vs. Semantics

Syntax

Form / structure of
expressions, statements,
and programs

Example:

whether `a = b + const` is
written correctly.

Correct structure?

Semantics

Meaning of expressions,
statements, and
programs

Example:

what happens after `a = b + 1`
executes.

Meaning as effect

Language description terminology

Term	Meaning	Example
Sentence	A string of characters in a language	<code>x = sum * 3;</code>
Language	A set of valid sentences	All valid Python programs, all valid Java statements
Lexeme	Lowest-level unit, such as <code>sum</code> , <code>*</code> , <code>begin</code>	<code>sum</code> , <code>*</code> , <code>begin</code> , <code>x</code> , <code>100</code>
Token	Category of lexemes, such as identifier, operator	<code>sum</code> → identifier; <code>*</code> → operator; <code>100</code> → integer literal
Recognizer	Determines whether a string belongs to a language	A compiler/parser checks whether <code>x = 3 + 5;</code> is valid syntax
Generator	Produces valid sentences of a language	A grammar rule generates valid expressions like <code>x + y</code> or <code>a * b</code>

Recognizers and generators

Recognizer

- Reads an input string.
- Answers: belongs to the language or not.
- Compiler parser is a recognizer.

Generator

- Produces valid sentences.
- Grammar rules generate structure.
- Useful for defining syntax.

Example:

input: `a = b + const`

output: valid?

Example:

`<stmt> → <var> = <expr>`

produces: `a = b + const`

BNF and Context-Free Grammars

- BNF(Backus-Naur Form) is a notation for describing programming language syntax.
- BNF is equivalent to context-free grammars.
- A grammar is a finite, non-empty set of rules.
- Rules have:
 - ✓ nonterminals, such as `<expr>`
 - ✓ terminals, such as `+`, `if`, identifier
 - ✓ a start symbol

Example:

```
<expr> → <term> + <term> | <term>  
<term> → <var> | const
```

Derivations

- A derivation is a repeated application of rules, starting with the start symbol and ending with a sentence (all terminal symbols)

```
<program> → <stmts>  
<stmts> → <stmt> | <stmt> ; <stmts>  
<stmt> → <var> = <expr>  
<var> → a | b | c | d  
<expr> → <term> + <term> | <term> - <term>  
<term> → <var> | const
```

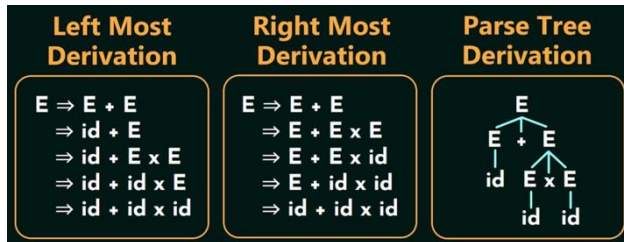
Grammar

```
<program>  
⇒ <stmts>  
⇒ <stmt>  
⇒ <var> = <expr>  
⇒ a = <expr>  
⇒ a = <term> + <term>  
⇒ a = <var> + <term>  
⇒ a = b + const
```

Derivation

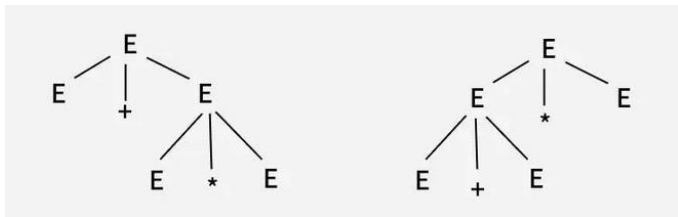
Derivations

- A leftmost derivation is one in which the leftmost nonterminal in each sentential form is the one that is expanded
- A derivation may be neither leftmost nor rightmost
- A parse tree shows the hierarchical structure of a derivation.



Ambiguity in Grammars

- A grammar is ambiguous if and only if it generates a sentential form that has two or more distinct parse trees
- Grammar: $E \rightarrow E + E \mid E * E \mid id$, We can create two parse tree from this grammar to obtain a string $id + id * id$



Real-world example: Ambiguous if-then-else

```
<if_stmt> → if (<logic_expr>) <stmt>  
          | if (<logic_expr>) <stmt> else <stmt>
```

```
if (A)  
    if (B) S1  
else S2
```

- Question: does else belong to the inner if or outer if?
- Most languages attach else to the nearest unmatched if.

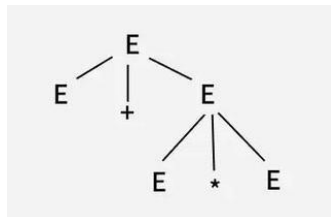
Removing Ambiguity in Grammars

- Grammar: $E \rightarrow E + E \mid E * E \mid id$
- Idea: put higher-precedence operations lower in the grammar.
- Separate the grammar into precedence levels:

$E \rightarrow E + T \mid T$ (low)

$T \rightarrow T * F \mid F$ (higher)

$F \rightarrow id$ * has **higher precedence** than +



Extended BNF

- **Extended BNF (EBNF)** is a more convenient version (shorter, cleaner, easier to understand) of **BNF** for writing grammar rules

EBNF Symbol	Meaning
[]	optional
(a b)	choice
{ }	repetition, zero or more times

BNF describes syntax using rules like:

```
<expr> → <expr> + <term> | <term>  
<term> → <term> * <factor> | <factor>
```

EBNF can write:

```
<expr> → <term> { (+ | -) <term> }  
<term> → <factor> { (* | /) <factor> }
```

Why CFGs are not enough

- Context-free grammars describe structure well.
- But they cannot express every programming language rule cleanly.
- Some rules require information about context, types, and declarations.

$x = y + 1$

Static semantics:

Rules that are not easily captured by CFGs but should be checked before running the program.

Questions CFGs do not answer:

- Was x declared?
- Is y numeric?
- Is the assignment type-correct?

Examples:

- Variables declared before use
- Compatible operand types
- Function arguments match parameters
- Return type is correct

Attribute grammars

- Attribute grammars extend CFGs by attaching information to grammar symbols.
 - ✓ Grammar symbols can carry attributes such as type, value, or scope.
 - ✓ Rules compute attributes and predicates check consistency.
 - ✓ Useful for type checking in compilers.

<var>.actual_type

**predicate:
type(left) = type(right)**

lookup(id)

Three approaches to semantics

Type	Main Idea	Best use
Operational semantics	Meaning is described by execution	teaching, manuals
Denotational semantics	Meaning is described by mathematical functions	rigor, language design
Axiomatic semantics	Meaning is described by logic assertions	program correctness proofs

Same program, three ways to define what it means.

Operational Semantics

- [Main idea] Operational semantics explains meaning by the state changes caused by execution.
- Good for teaching and language manuals.

Program state before

```
x = 3  
y = 5
```

Statement

```
x = x + y
```

Program state after

```
x = 8  
y = 5
```


Denotational Semantics

- [Main idea] Map each language construct to a mathematical object.
- Represent program state as values of variables.
- Define expressions and statements as functions over states.
- Very rigorous, but often heavy for language users.

```
state s = {<x, 3>, <y, 5>}
```

```
VARMAP(x, s) = 3
```

```
M_expr(x + y, s) = 8
```

mathematically heavy, so
only introduce the idea
briefly:

*A variable lookup function
returns the current value of
a variable.*

Axiomatic Semantics

- Axiomatic semantics uses logic to reason about program correctness.
- [Main idea] prove the program satisfies its specification.

$\{P\}$ statement $\{Q\}$

P = precondition

Q = postcondition

Examples:

```
{ b > 0 }
```

```
a = b + 1
```

```
{ a > 1 }
```

Loop Invariants

- A loop invariant is a condition that remains true:
 - ✓ before the loop starts
 - ✓ after each iteration
 - ✓ when the loop ends
- This is useful for proving loop correctness.

Loop invariant: sum contains the total of processed elements.

Loop invariant: After k iterations,
 $\text{sum} = 1 + 2 + \dots + k$

```
while (i < n) {  
    sum = sum + A[i];  
    i = i + 1;  
}
```

```
sum = 0  
for i = 1 to n:  
    sum = sum + i
```

Summary

Concept	Main purpose or idea
BNF / CFG	Formal tools for describing syntax
Derivation / parse tree	How grammar rules generate structure
Ambiguity	Multiple parse trees; must be avoided
EBNF	Compact grammar notation
Static semantics	Type/declaration rules beyond CFG
Semantics	Operational, denotational, and axiomatic views of meaning

Formal syntax tells us what programs look like;

Formal semantics tells us what programs mean.